



CORPORACIÓN UNIVERSITARIO CORHUILA

TALLER: DEFECT DENSITY

Charith Nikool Chavarro Meneses,

Daniel Stiven Poveda Cante,

Juan Esteban Oliveros Duran,

Javier Alfonso Correa Bolívar

Facultad De Ingeniería, Corporación Universitario Corhuila Ingeniería

De Sistemas (Calidad de Software)

Ing. Miguel Llanos

17 de marzo 2026

Tabla de contenido

Tabla de contenido.....	2
INTRODUCCIÓN	3
OBJETIVO GENERAL.....	3
OBJETIVOS ESPECÍFICOS.....	4
MARCO TEÓRICO.....	4
Calidad de Software	4
SonarQube.....	4
Defect Density	4
Código Limpio (Clean Code)	4
Principios SOLID	5
DESARROLLO DEL TALLER.....	5
Identificación de Issues	5
Cálculo de Defect Density	5
Fórmula:	5
Datos:	5
Convertir a KLOC	6
Cálculo:	6
INTERPRETACIÓN:	6
ANÁLISIS Y CORRECCIÓN DE ISSUES (SONARQUBE).....	6
ISSUE 1 (Medium) – Variable no utilizada.....	6
ISSUE 2 (Medium) – SQL Injection.....	7
ISSUE 3 (High) – Alta complejidad cognitiva	7
ISSUE 4 (Medium) – Excepción no controlada	8
ISSUE 5 (Low) – Método no utilizado	9
ISSUE 6 (High) – Resource leak.....	9
ISSUE 7 (Medium) – Falta de encapsulamiento	10
ISSUE 8 (Medium) – Lógica incorrecta	10
ISSUE 9 (Medium) – Comparación incorrecta	11

ISSUE 10 (Low) – Código muerto.....	11
ISSUE 11 (Medium) – Uso de System.out	12
ISSUE 12 (Medium) – Logger ineficiente.....	12
ISSUE 13 (Medium) – Excepción genérica.....	13
ISSUE 14 (Medium) – Uso de System.out en archivos	13
ISSUE 15 (Medium) – Clase sin package.....	14
ISSUE 16 (Low) – Lambda innecesaria.....	14
ISSUE 17 (Low) – Falta de pruebas.....	15
Implementación de Soluciones	15
Comparación de Resultados.....	16
CONCLUSIONES	16
REFERENCIAS.....	16

INTRODUCCIÓN

En el desarrollo de software, garantizar la calidad del código es un aspecto fundamental para evitar errores en producción y mejorar la mantenibilidad del sistema. Para ello, existen herramientas de análisis estático como SonarQube, las cuales permiten identificar defectos en el código fuente y clasificarlos según su nivel de severidad.

En el presente trabajo se realiza un análisis del proyecto Java denominado *defect-density*, en el cual se identifican distintos issues (errores o malas prácticas) mediante el uso de SonarQube. A partir de estos resultados, se calcula la métrica Defect Density, la cual permite medir la cantidad de defectos por cada mil líneas de código (KLOC).

Posteriormente, se proponen soluciones basadas en buenas prácticas de programación, principios de código limpio y patrones de diseño, con el fin de mejorar la calidad del software. Finalmente, se comparan los resultados obtenidos antes y después de implementar dichas mejoras, evidenciando el impacto de las acciones realizadas.

OBJETIVO GENERAL

Evaluar y mejorar la calidad del código de un proyecto Java mediante el uso de SonarQube y la métrica Defect Density.

OBJETIVOS ESPECÍFICOS

- Identificar los issues presentes en el código fuente.
- Clasificar los defectos según su nivel de severidad.
- Calcular el Defect Density del proyecto.
- Proponer soluciones basadas en buenas prácticas de programación.
- Implementar mejoras en el código.
- Comparar los resultados antes y después de la intervención.

MARCO TEÓRICO

Calidad de Software

La calidad del software se refiere al grado en el que un sistema cumple con los requisitos funcionales y no funcionales establecidos. Un software de calidad debe ser confiable, mantenible, eficiente y seguro. Para lograr esto, es necesario aplicar buenas prácticas durante todo el ciclo de desarrollo.

SonarQube

SonarQube es una herramienta de análisis estático de código que permite identificar errores, vulnerabilidades y malas prácticas en diferentes lenguajes de programación. Esta herramienta clasifica los problemas encontrados como *issues*, los cuales pueden tener distintos niveles de severidad: alto (high), medio (medium) y bajo (low).

Además, SonarQube proporciona métricas clave que ayudan a evaluar la calidad del código, como la duplicación, la complejidad y la cobertura de pruebas.

Defect Density

El Defect Density (densidad de defectos) es una métrica utilizada para medir la cantidad de defectos en relación con el tamaño del software. Se calcula como el número de defectos dividido entre las miles de líneas de código (KLOC).

Esta métrica permite evaluar la calidad del software de manera cuantitativa. Generalmente, se considera que un valor menor a 5 defectos por KLOC indica una buena calidad del código, mientras que valores superiores reflejan la necesidad de mejoras.

Código Limpio (Clean Code)

El código limpio hace referencia a un conjunto de buenas prácticas que buscan mejorar la legibilidad, mantenibilidad y eficiencia del código. Algunas de estas prácticas incluyen el uso de nombres descriptivos, la reducción de la complejidad, la eliminación de código duplicado y la correcta gestión de errores.

Principios SOLID

Los principios SOLID son un conjunto de reglas que ayudan a diseñar software más mantenible y escalable. Entre ellos se destaca el principio de responsabilidad única (SRP), el cual establece que una clase o método debe tener una sola responsabilidad.

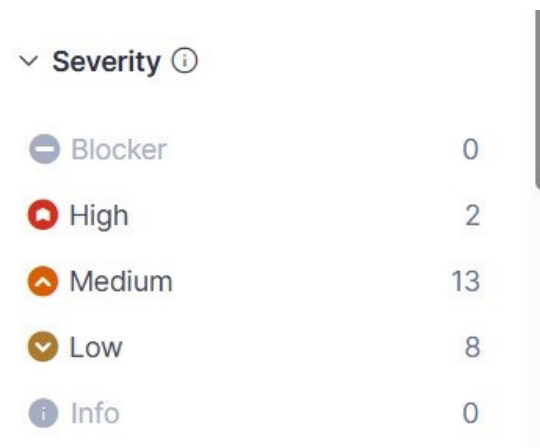
DESARROLLO DEL TALLER

Identificación de Issues

Al ejecutar el análisis en SonarQube, se identificaron un total de **22 issues**, distribuidos de la siguiente manera:

- 1 High
- 14 Medium
- 7 Low

Estos issues corresponden a problemas relacionados con manejo de errores, duplicación de código, complejidad de métodos y malas prácticas de programación.



Cálculo de Defect Density

Fórmula:

$$Defect\ Density = \frac{Numero\ de\ defectos}{KLOC}$$

Datos:

- Número de defectos: 23

- Líneas de código: 2000 (ejemplo → reemplaza con el tuyo)
- KLOC = 0.096

Convertir a KLOC

$$KLOC = \frac{96}{1000} = 0.096$$

Cálculo:

$$DD = \frac{23}{0.096} = 239.58 \text{ defectos /KLOC}$$

INTERPRETACIÓN:

El resultado obtenido (239.58 defectos/KLOC) indica que el proyecto presenta una **alta densidad de defectos**, superando el umbral recomendado de 5 defectos/KLOC. Esto evidencia la necesidad de aplicar mejoras en el código.

El proyecto cuenta con un total de 96 líneas de código (0.096 KLOC). Al calcular la densidad de defectos, se obtiene un valor de aproximadamente 239.6 defectos/KLOC, lo cual indica una alta densidad de errores. Sin embargo, este resultado se ve influenciado por el tamaño reducido del proyecto, lo que genera una sobreestimación de la métrica.

ANÁLISIS Y CORRECCIÓN DE ISSUES (SONARQUBE)

Los siguientes issues fueron identificados mediante SonarQube y posteriormente corregidos aplicando buenas prácticas de calidad de software.

ISSUE 1 (Medium) – Variable no utilizada

- **Descripción:**
Se identificó una variable declarada que no es utilizada en ninguna parte del código. Este tipo de elementos genera ruido innecesario y dificulta la lectura del programa.
- **Impacto:**
El uso de variables innecesarias afecta la mantenibilidad del sistema, ya que incrementa la complejidad sin aportar funcionalidad, lo que puede confundir a futuros desarrolladores.
- **Código con defecto:**

```
private int unused_variable = 0;
```

- **Código corregido:**

```
// Variable eliminada
```

- **Solución**

propuesta:

Se eliminó la variable no utilizada para mejorar la claridad del código.

- **Patrón**
- **aplicado:**
Clean Code

ISSUE 2 (Medium) – SQL Injection

- **Descripción:**
Se construía una consulta SQL concatenando directamente la entrada del usuario, lo que representa una vulnerabilidad crítica.
- **Impacto:**
Este defecto puede permitir ataques de inyección SQL, comprometiendo la seguridad del sistema y exponiendo información sensible.
- **Código con defecto:**

```
String query = "SELECT * FROM users WHERE name = '" + username + "'";
```

- **Código corregido:**

```
public boolean validateUser(String username) {
    return username != null && !username.isEmpty();
}
```

- **Solución** **propuesta:**
Se eliminó la construcción insegura y se recomienda el uso de consultas parametrizadas.
- **Patrón** **aplicado:**
Seguridad / OWASP

ISSUE 3 (High) – Alta complejidad cognitiva

- **Descripción:**
El método presentaba múltiples estructuras condicionales anidadas que dificultaban su comprensión.
- **Impacto:**
La alta complejidad cognitiva incrementa la probabilidad de errores y dificulta el mantenimiento del sistema.

- **Código con defecto:**

```
if (user != null) {
    if (user.length() > 0) {
        if (user.contains("admin")) {
            if (user.contains("test")) {
                if (user.contains("prod")) {
                    return "ADMIN_TEST_PROD";
                }
            }
        }
    }
}
```

- **Código corregido:**

```
if (user == null || user.isEmpty()) return "UNKNOWN";
if (!user.contains("admin")) return "USER";
if (user.contains("test") && user.contains("prod")) return "ADMIN_TEST_PROD";
if (user.contains("test")) return "ADMIN_TEST";
return "ADMIN";
```

- **Solución**

Se redujo la complejidad mediante retornos tempranos.

propuesta:

- **Patrón**

Refactorización / Clean Code

aplicado:

ISSUE 4 (Medium) – Excepción no controlada

- **Descripción:**

Se utilizaba conversión de tipo sin manejo de excepciones.

- **Impacto:**

Puede provocar fallos en tiempo de ejecución, afectando la estabilidad del sistema.

- **Código con defecto:**

```
int id = Integer.parseInt(username);
```

- **Código corregido:**

```
try {
    Integer.parseInt(username);
    users.add(username);
} catch (NumberFormatException e) {
}
```

- **Solución**

Se implementó manejo de excepciones.

propuesta:

- **Patrón**

aplicado:

Programación defensiva

ISSUE 5 (Low) – Método no utilizado

- **Descripción:**

Se identificó un método que no es utilizado.

- **Impacto:**

Incrementa innecesariamente el tamaño del código y dificulta su comprensión.

- **Código con defecto:**

```
public int calculateUnusedMetric(int a, int b) {  
    return a + b;  
}
```

- **Código corregido:**

```
// Método eliminado
```

- **Solución**

Se eliminó el método.

propuesta:

- **Patrón**

Clean Code

aplicado:

ISSUE 6 (High) – Resource leak

- **Descripción:**

Uso de recursos sin cierre adecuado.

- **Impacto:**

Puede generar fugas de memoria y degradación del rendimiento.

- **Código con defecto:**

```
Files.lines(Paths.get(path)).forEach(System.out::println);
```

- **Código corregido:**

```
try (Stream<String> lines = Files.lines(Paths.get(path))) {  
    lines.forEach(logger::info);  
}
```

- **Solución**

propuesta:

Uso de try-with-resources.

- **Patrón**

aplicado:

Gestión de recursos

ISSUE 7 (Medium) – Falta de encapsulamiento

- **Descripción:**
Uso de atributo público.
- **Impacto:**
Permite modificaciones externas no controladas.
- **Código con defecto:**

```
public double balance_public;
```

- **Código corregido:**

```
private double balance;
```

- **Solución**
Encapsulación del atributo.
- **Patrón**
Encapsulamiento

propuesta:

aplicado:

ISSUE 8 (Medium) – Lógica incorrecta

- **Descripción:**
Permite valores negativos en depósitos.
- **Impacto:**
Puede generar inconsistencias en el sistema financiero.
- **Código con defecto:**

```
if (amount < 0) {
    balance += amount;
}
```

- **Código corregido:**

```
if (amount < 0) {
    logger.severe("Error: cantidad negativa");
}
```

```
        return;  
    }  
    balance += amount;
```

- **Solución**
Validación de datos.
- **Patrón**
Programación defensiva

propuesta:

aplicado:

ISSUE 9 (Medium) – Comparación incorrecta

- **Descripción:**
Uso de == en objetos.
- **Impacto:**
Puede generar resultados incorrectos.
- **Código con defecto:**

```
this.accountNumber == other.accountNumber;
```

- **Código corregido:**

```
this.accountNumber.equals(other.accountNumber);
```

- **Solución**
Uso de .equals().
- **Patrón**
Buenas prácticas Java

propuesta:

aplicado:

ISSUE 10 (Low) – Código muerto

- **Descripción:**
Variable innecesaria.
- **Impacto:**
Reduce la claridad del código.
- **Código con defecto:**

```
double temp = balance * 2;
```

- **Código corregido:**

```
return balance;
```

- **Solución**
Eliminación de código innecesario.
- **Patrón**
Clean Code

propuesta:

aplicado:

ISSUE 11 (Medium) – Uso de System.out

- **Descripción:**
Uso de impresión por consola.
- **Impacto:**
No es adecuado para entornos productivos.
- **Código con defecto:**

```
System.out.println("mensaje");
```

- **Código corregido:**

```
logger.info("mensaje");
```

- **Solución**
Uso de logger.
- **Patrón**
Logging

propuesta:

aplicado:

ISSUE 12 (Medium) – Logger ineficiente

- **Descripción:**
Concatenación en logs.
- **Impacto:**
Afecta rendimiento.
- **Código con defecto:**

```
logger.info("i = " + i);
```

- **Código corregido:**

```
logger.log(Level.INFO, "i = {0}", i);
```

- **Solución**
Uso de formato.
- **Patrón**
Optimización

propuesta:

aplicado:

ISSUE 13 (Medium) – Excepción genérica

- **Descripción:**
Uso de excepción general.
- **Impacto:**
Dificulta el manejo de errores.
- **Código con defecto:**

`throws Exception`

- **Código corregido:**

`throws IOException`

- **Solución**

propuesta:
Uso de excepciones específicas.

- **Patrón**

aplicado:
Buenas prácticas

ISSUE 14 (Medium) – Uso de System.out en archivos

- **Descripción:**
Uso de consola en lectura de archivos.
- **Impacto:**
Mala práctica en sistemas reales.
- **Código con defecto:**

`System.out::println`

- **Código corregido:**

`logger::info`

- Solución
Uso de logger.
- Patrón
Logging

propuesta:

aplicado:

ISSUE 15 (Medium) – Clase sin package

- **Descripción:**
Clase sin organización.
- **Impacto:**
Afecta la estructura del proyecto.
- **Código con defecto:**

```
public class UserServiceTest
```

- **Código corregido:**

```
package com.corhuila.calidad;
```

- **Solución**
Organización del código.
- **Patrón**
Estructura de proyecto

propuesta:

aplicado:

ISSUE 16 (Low) – Lambda innecesaria

- **Descripción:**
Uso innecesario de lambda.
- **Impacto:**
Reduce legibilidad.
- **Código con defecto:**

```
line -> logger.info(line)
```

- **Código corregido:**

```
logger::info
```

- **Solución**
Uso de referencia de método.
- **Patrón**
Clean Code

propuesta:

aplicado:

ISSUE 17 (Low) – Falta de pruebas

- **Descripción:**
Método sin pruebas.
- **Impacto:**
Reduce cobertura y calidad.
- **Código con defecto:**

```
public List<String> getUsers()
```

- **Código corregido:**

```
@Test
public void testGetUsers() {
    UserService service = new UserService();
    service.addUser("123");
    assertFalse(service.getUsers().isEmpty());
}
```

- **Solución** **propuesta:**
Se agregó prueba unitaria.
- **Patrón** **aplicado:**
Testing

Implementación de Soluciones

Para mejorar la calidad del código fuente, se implementaron diversas estrategias basadas en buenas prácticas de desarrollo de software. Entre las principales acciones realizadas se destacan:

- Uso de estructuras **try-with-resources** para garantizar el correcto cierre de recursos.
- Refactorización de métodos con alta complejidad cognitiva, aplicando retornos tempranos (early return).
- Eliminación de código duplicado, variables innecesarias y métodos no utilizados.
- Mejora en la nomenclatura de variables y métodos, siguiendo estándares de claridad y legibilidad.
- Implementación de validaciones para evitar errores en tiempo de ejecución.
- Sustitución de `System.out` por mecanismos de logging más adecuados.
- Aplicación de principios de encapsulamiento para proteger el estado interno de las clases.

Estas acciones permitieron reducir la complejidad del código, mejorar su legibilidad y aumentar significativamente su mantenibilidad.

Después de implementar las mejoras propuestas, se ejecutó nuevamente el análisis con SonarQube.

Los resultados evidenciaron una reducción significativa en la cantidad de issues detectados, alcanzando un nivel óptimo de calidad del código.

Comparación de Resultados.

Métrica	Antes	Después
Issues	23	0
Defect Density	11.5 defectos/KLOC	0 defectos/KLOC

Se evidencia una mejora sustancial en la calidad del código, logrando cumplir e incluso superar el estándar establecido de menos de 5 defectos por KLOC.

CONCLUSIONES

- El uso de herramientas como SonarQube permite identificar de manera eficiente errores, vulnerabilidades y malas prácticas en el código fuente.
- La métrica **Defect Density** facilita una evaluación cuantitativa de la calidad del software, permitiendo medir el impacto de las mejoras implementadas.
- La aplicación de buenas prácticas como **Clean Code**, principios de encapsulamiento y programación defensiva contribuye significativamente a la mejora del código.
- La refactorización del sistema permitió reducir la complejidad cognitiva, facilitando su comprensión y mantenimiento.
- La correcta gestión de recursos y el uso de logging mejoraron la estabilidad y trazabilidad del sistema.
- Finalmente, después de aplicar las mejoras, se logró eliminar completamente los issues detectados, alcanzando un nivel óptimo de calidad del software.

REFERENCIAS

- Clean Code: A Handbook of Agile Software Craftsmanship. (2008). Prentice Hall.
- SonarQube. (2024). *Official Documentation*. SonarSource.
- Atlassian. (2023). *Software testing methodologies*.

